

Supply Chain Demand Forecasting Service

Development rationale, implementation guide, results and handover

Repository: garietan3066/demand-forecasting-service

Snapshot: 24 September 2026 | local main at b0dd1e2

Audience: project owner, technical reviewer and portfolio interviewer

What has actually been completed?

A reproducible next-day sales forecasting pipeline has been developed, evaluated and packaged. It includes audited real data, chronological cross-validation, baseline and model comparisons, a frozen final evaluation, historical feature generation for serving, and verified native model loading.

This is not yet a deployed microservice. PostgreSQL, operational ingestion, authenticated FastAPI endpoints, container deployment and replenishment simulation remain unfinished. The reusable forecasting and packaging components are the completed foundation.

Evidence	Recorded outcome
Final evaluation	200 store-product series; 1,400 predictions over seven days
Selected method	50% LightGBM + 50% preceding-seven-day mean
Final WAPE	36.91%, versus 38.16% for the seven-day baseline
Serving verification	All 1,400 predictions reproduced with zero numerical difference
Tests	Latest user-provided run: 141 passed; not rerun to author this PDF
Git state at inspection	main synchronized with origin/main; annotation fix merged in PR #14

How to use this guide

Read pages 2-5 for the reasoning and measured results. Pages 6-15 explain the reusable code and every experiment script. Pages 16-19 cover application scaffolding, tests, configuration and evidence files. Pages 20-21 explain safe resumption and the remaining work.

The explanations are based on the checked-out source and committed reports. Short code excerpts illustrate the implementation; they are not replacement files. No training, installation, model rebuild or repository commit was performed to write this document.

Problem and development choices

Business intention versus measured target

The intended user is an inventory planner deciding what stock to prepare. A forecast is one input to that decision; inventory on hand, incoming deliveries, supplier lead times, service targets and spoilage also matter. A predicted sales value is not automatically an order quantity.

Demand is what customers want to buy. Sales are what they actually purchase. If demand is 100 and available stock is 60, sales may be 60; sales alone do not reveal the missing 40. Stockout indicators help identify constrained observations but do not directly measure lost demand.

For that reason, the first release predicts **observed next-day sales in normalized dataset units**. It does not claim to recover true demand or original physical-unit quantities. The project name describes its supply-chain direction; the precise evaluated target is narrower.

Why the development order changed

Decision	Reason
Inspect data before final API design	Identifiers, units, dates and available history determine a workable input contract.
Build baselines before complex models	A simple benchmark shows whether machine learning adds measurable value.
Use multiple chronological folds	One favorable time window can give a misleading impression.
Keep one final period reserved	Model selection consumes development evidence; final scoring needs a separate period.
Build serving features before the database	Establish the correct historical inputs before implementing their storage and retrieval.
Package before deployment	The service should load a verified artifact, not retrain during startup.

What the project demonstrates

Data science: target definition, temporal validation, feature engineering, honest model comparison and error diagnosis.

ML engineering: reusable modules, explicit contracts, repeatable environments, integrity checks, automated tests and artifact parity.

Still to demonstrate: database operations, secure API behavior, deployment, monitoring and recovery.

Dataset, subset and boundaries

FreshRetailNet-50K was chosen because it contains daily retail sales and hourly stockout information. That supports the distinction between sales and potentially constrained demand better than sales-only data. It is fresh-retail data, not manufacturing-component data.

Property	Project interpretation
Training source	4,500,000 daily rows; 50,000 store-product series; 90 days
Training dates	2024-03-28 through 2024-06-25
Development subset	200 deterministically selected series; 18,000 daily rows
Model-eligible full training rows	12,000 after 30 initial history rows per series are excluded
Final subset	2024-06-26 through 2024-07-02; $200 \times 7 = 1,400$ rows
Core key	store_id + product_id + dt
Target	sale_amount: normalized observed sales
Retrospective diagnostic	stock_hour6_22_cnt: stockout-hour count for the 06:00-22:00 window

How series selection avoids cherry-picking

The baseline script takes identifiers present on the first training date, hashes the string freshretail-v1:store:product with SHA-256, sorts by that hash and takes 200 pairs. Selection does not depend on sales magnitude or forecasting errors. The resulting pair list is recorded and checked in later experiments.

Why original data and predictions stay outside Git

Raw/cache data and generated row-level predictions live under ignored data/. Code and experiment reports carry the provenance and conclusions. Parquet is used for processed tabular results; Arrow batches allow cache data to be read without expanding all hourly arrays into a huge DataFrame.

Limits that must remain visible

Ninety days do not support claims about annual seasonality. Normalized values cannot be called units sold without a verified inverse transformation. Stockout flags can coexist with positive hourly sales; within-hour timing is not fully resolved. Discounts above one were observed and are not used in the current predictors.

Dataset attribution and CC BY 4.0 are recorded in docs/project_scope.md. The source URL is listed on page 21. This PDF does not independently recertify the publisher license or data authenticity.

Time boundaries and leakage prevention

Leakage occurs when a prediction uses information unavailable when it would have been issued. A random row split can put future observations into training while earlier dates remain in validation. For a panel dataset, dates must be split across all series together.

Fold	Training ends	Validation	Usable training / validation rows
1	May 14	May 15-28	3,600 / 2,800
2	May 28	May 29-June 11	6,400 / 2,800
3	June 11	June 12-25	9,200 / 2,800

Every fold starts from the same March 28 history origin. A fresh model is fitted per fold; the training window expands. Validation contains 14 days x 200 series. Pooled validation covers 8,400 nonoverlapping observations.

Rolling one-day forecasting

For June 26, use observations through June 25. Once June 26 finishes and its actual sales are available, they can enter the history for June 27. The fitted model remains fixed during the final evaluation. Updating historical inputs is not the same as retraining model parameters.

June 25 history -> predict June 26 -> observe June 26
June 26 history -> predict June 27 -> observe June 27

This is not a seven-day forecast issued entirely on June 25. Earlier validation actuals are allowed only because the task is rolling one day ahead. The project assumes previous-day observations arrive in time; actual reporting delays are not established by this dataset.

Concrete protections in the code

Group shifts and rolling windows by store-product pair; shift before averaging; reject duplicates and missing daily rows; fit categories and preprocessing on training only; pass predictors without target sales; exclude realized target-date stockouts and weather; enforce prediction dates after the training cutoff.

Tests alter current and future sales and assert that earlier features or predictions stay unchanged. These are evidence of the implemented boundary, not a universal proof against every possible future leakage bug.

Results and model-selection reasoning

Candidate	Development WAPE	Final WAPE
50/50 blend	34.39%	36.91%
Logistic + Ridge	35.03%	Not included
Ridge	35.13%	Not included
Seven-day mean	35.60%	38.16%
LightGBM	35.62%	37.28%
Linear regression	36.91%	Not included

What WAPE actually means

$WAPE = \frac{\sum(\text{abs}(\text{prediction} - \text{actual}))}{\sum(\text{actual})}$

$MAE = \text{mean}(\text{abs}(\text{prediction} - \text{actual}))$

$\text{bias} = \text{mean}(\text{prediction} - \text{actual})$

A WAPE of 36.91% means absolute errors sum to 36.91% of the sum of observed sales, in the same measurement scale. It is not 63.09% accuracy, and individual errors can be much larger or smaller. Positive and negative errors do not cancel in WAPE; they do cancel in signed bias.

The final blend improves on 38.16% baseline WAPE by about **1.25 percentage points**, or **3.3% less total absolute error** on identical rows. This is a modest improvement, not a large forecasting breakthrough. The blend won each development fold; final scoring retained the method selected in advance.

Why not keep searching for 10-15%?

Similar model scores do not prove an irreducible error floor. Additional information, broader training coverage or different objectives might help. The current decision was to complete the engineering workflow rather than promise a target. The final week has now been inspected, so later model selection cannot treat it as a fresh independent holdout.

A seven-day final period is limited evidence. Neither WAPE nor the small mean bias establishes inventory savings, service-level performance, or accuracy against unobserved demand. Those require separate evaluation.

Reusable core: baselines and metrics

demand_forecasting/constants.py

Purpose and data flow. Defines KEYS and baseline names used across modules.

How the code works. KEYS is store_id and product_id; BASELINES names yesterday, last_week and mean_previous_7_days.

Why this design. One shared definition reduces accidental disagreement in joins, grouping and report columns.

demand_forecasting/baselines.py

Purpose and data flow. predict_baselines(frame) takes daily observations and returns sorted observations plus three forecast columns.

How the code works. Copies the input; parses dt; sorts by keys and date; resets the index; rejects nulls, duplicates, gaps, nonfinite and negative sales. Grouped shift(1), shift(7), and shift(1).rolling(7).mean() create predictions.

Why this design. Sorting makes positional history meaningful; checking gaps prevents the previous row from being mistaken for the previous calendar day. Grouping prevents one product borrowing another product's history.

```
grouped = frame.groupby(KEYS).sale_amount
frame["yesterday"] = grouped.shift(1)
# Seven-day mean uses yesterday and the six preceding days.
```

demand_forecasting/metrics.py

Purpose and data flow. metrics(actual, predicted) returns row count, WAPE, MAE and mean signed error.

How the code works. Converts inputs to NumPy arrays, rejects nonfinite values and calculates errors as predicted minus actual. Empty inputs return null metrics; zero total actual sales gives WAPE=None.

Why this design. Undefined WAPE is not disguised as zero error. Shared metric logic ensures model comparisons use the same arithmetic. Callers remain responsible for aligning matching, one-dimensional observations.

demand_forecasting/__init__.py

Purpose and data flow. Marks the reusable forecasting package.

How the code works. Contains package-level description rather than a training job.

Why this design. Importing a package should not launch training or mutate data. The package separates reusable functions from experiment scripts.

Reusable core: splits and evaluation

demand_forecasting/validation.py

Purpose and data flow. `expanding_date_folds(frame)` returns `TemporalFold` objects containing date bounds and row positions.

How the code works. Checks a complete daily panel, timezone-naive dates and unique series/date keys. Applies scikit-learn `TimeSeriesSplit` to sorted unique dates, not to interleaved panel rows. Maps each date window back with `np.flatnonzero`.

Why this design. All stores and products share the same temporal cutoff. Returning positions lets callers use `iloc` even if the original `DataFrame` has shuffled or nonstandard index labels.

```
splitter = TimeSeriesSplit(
    n_splits=3, test_size=14, gap=0
)
# split(unique_dates), then map dates back to panel rows
```

Why gap=0 is acceptable here: this task uses observed daily targets with strictly lagged inputs and assumes previous-day observations are available. A reporting delay or overlapping multi-day target would require reconsidering the split boundary.

demand_forecasting/evaluation.py

Purpose and data flow. `evaluate_baselines(frame)` produces a fold report and combined validation predictions.

How the code works. Builds baseline predictions first, splits that same sorted frame, selects each validation window, checks history availability and calculates per-method metrics. Adds fold IDs, concatenates results, rejects overlaps and calculates pooled scores.

Why this design. Using the same frame prevents prediction/index misalignment. Pooled WAPE is computed from all errors and actuals, rather than naively averaging fold percentages with different sales totals.

DataFrame operations in plain language

`copy()` prevents surprising changes to a caller's data. `groupby(KEYS)` keeps each series separate. `iloc` chooses rows by position. `concat` stacks windows. `merge(validate="one_to_one")` rejects joins that would multiply predictions. These operations support correctness; they are not cosmetic coding choices.

Reusable core: feature calculations

demand_forecasting/features.py

Purpose and data flow. `build_training_features(frame)` returns validated historical rows, model features and `history_ready`.

How the code works. Reuses baseline validation and sorted history; computes `lag_1`, `lag_7`, `lag_14`, `lag_30`, `mean_7`, `mean_14`, `std_7` and `day_of_week`. Store and product IDs are categorical features. Removes baseline-only columns before returning.

Why this design. One explicit `FEATURE_COLUMNS` list defines model order. Keeping incomplete initial rows visible preserves history while allowing callers to select eligible training rows deliberately.

Feature	Information available at forecast date t
<code>lag_1 / lag_7 / lag_14 / lag_30</code>	Sales at t-1, t-7, t-14 and t-30 within the same series
<code>mean_7 / mean_14</code>	Mean of the preceding 7 or 14 daily observations
<code>std_7</code>	Population standard deviation of the preceding 7 observations (ddof=0)
<code>day_of_week</code>	Known calendar weekday: Monday 0 through Sunday 6
<code>store_id / product_id</code>	Identifiers encoded as categorical values, not ordered magnitudes

```
grouped.transform(
    lambda sales: sales.shift(1)
        .rolling(14, min_periods=14).mean()
)
```

The shift occurs **before** the rolling calculation. Without it, today's observed target would contribute to the feature used to predict today. `min_periods` prevents partial windows from silently changing the meaning of the feature.

For each 90-day series, the first 30 rows lack `lag_30`. Therefore, 60 rows per series are eligible: $200 \times 60 = 12,000$. Earlier rows still supply the required historical observations.

Not implemented: cyclical sine/cosine calendar encoding, annual seasonality, future weather, promotions and historical-stockout predictors. They were discussed as possible extensions, not part of the released feature contract.

Reusable core: fitting and blending

demand_forecasting/modeling.py

Purpose and data flow. FittedForecaster bundles a model, category mappings, known pairs and training cutoff. `feature_matrix()`, `fit_forecaster()` and `predict_forecaster()` share a strict input path.

How the code works. Training checks complete history and nonnegative finite targets, learns categories only from training, and fits a fresh LGBMRegressor. Prediction rejects dates at or before `training_end` and unknown pairs, validates features, predicts, then clips negative outputs to zero.

Why this design. The object stores more than weights: it carries the interpretation and supported population of those weights. Native categories do not imply that IDs are continuous quantities.

Parameter	Selected setting / purpose
<code>objective</code>	regression: squared-error training objective
<code>n_estimators / learning_rate</code>	200 / 0.05: number and contribution of boosted trees
<code>num_leaves / min_child_samples</code>	15 / 30: limits on tree complexity and small leaves
<code>reg_lambda</code>	1.0: regularization
<code>random_state / n_jobs</code>	42 / 2: fixed seed and bounded training parallelism
<code>deterministic / force_col_wise</code>	True / True: controlled training configuration

These are fixed experimental settings, not proven optimal hyperparameters. Training uses squared error while reporting WAPE/MAE; testing a different objective would be a new experiment, not a documentation correction.

demand_forecasting/ensemble.py

Purpose and data flow. `equal_weight_blend(baseline, model)` returns a 50/50 average.

How the code works. Requires matching one-dimensional shapes and finite, nonnegative inputs; returns 0.5 times each vector.

Why this design. The weight was fixed rather than searched. Combining different errors improved development performance, but averaging does not guarantee improvement in other periods.

Type annotation: `model: lgb.LGBMRegressor | lgb.Booster` permits a trained sklearn wrapper or a native loaded Booster. The annotation does not select an algorithm, change weights or retrain anything. PR #14 saved this intended clarification.

Reusable core: tomorrow without tomorrow's sales

demand_forecasting/serving.py

Purpose and data flow. `build_next_day_features(history, forecast_date)` returns one predictor row per supplied series, without returning a target column.

How the code works. Selects the required historical columns; validates daily dates; rejects observations on or after `forecast_date`. Reuses historical feature validation, requires at least 30 consecutive days per pair, and requires every series to end yesterday.

Why this design. A real caller cannot supply tomorrow's target. The service needs a historical-data adapter, not a request that asks the caller to manufacture trusted lag features.

Why there is an internal zero placeholder

The adapter appends one row per series at the requested date with `sale_amount=0.0`, then invokes the existing shifted feature builder. The placeholder allows the historical routine to create the next row. Since all sales-derived features use previous observations, this zero does not enter the target-date predictors. It is removed from the returned columns.

This is an implementation device, not an assumption that tomorrow has zero sales. Tests compare returned serving features with historical features for multiple dates, providing evidence that the two paths agree.

```
history through June 25
-> build_next_day_features(history, "2024-06-26")
-> predict_forecaster(loader_model, features)
-> equal_weight_blend(features.mean_7, predictions)
```

Important boundaries

The adapter builds features for pairs supplied by its caller. The loaded model checks whether those pairs were supported during training. Later database code must ensure all requested pairs are retrieved; the adapter alone cannot know that a caller accidentally omitted a requested series.

The current model predicts known-series next-day sales. It does not implement cold-start inference or a fixed-origin multi-day forecast. Longer histories are accepted, but missing internal dates and stale final dates are rejected.

Reusable core: artifact integrity and loading

demand_forecasting/artifacts.py

Purpose and data flow. `file_sha256()` calculates digests; `export_package()` writes native `model.txt` and `metadata.json`; `load_package()` verifies and loads the package.

How the code works. Export refuses an existing directory and records features, categories, known pairs, cutoff, blend settings, dependency versions and provenance. Load verifies metadata bytes against a separately supplied expected hash before parsing JSON, checks the contract and versions, then verifies model bytes before constructing a Booster.

Why this design. A model needs a reproducible interpretation, not just a weight file. Native LightGBM serialization avoids Python pickle object deserialization in this release path.

Verification order matters

```
trusted expected metadata hash
-> verify metadata.json bytes
-> validate contract and dependency versions
-> verify model.txt bytes against metadata
-> load those same bytes
-> verify feature and category order
```

The loader validates daily training cutoff, integer pair identifiers, duplicate pairs, category membership and category order. It restricts resolved package-file paths to the package directory and checks feature order against the application contract.

What hashes do and do not establish

Hashes detect changes relative to a trusted expected value. If an attacker can replace both the package and its trusted release record, matching hashes alone do not provide authenticity. Deployment must protect that record. Native parsing is also not a reason to accept arbitrary model uploads.

The current loader requires exact versions for LightGBM, NumPy, pandas and scikit-learn. This is a conservative compatibility rule; it does not independently compare every source-file provenance hash with the running application. Source hashes provide traceability, while tests and controlled releases establish behavioral compatibility.

If export fails partway through, a partial directory may remain; it is not a valid release. Export does not overwrite it automatically. The older `artifacts/model.joblib` is a separate prototype artifact, not the selected native release.

Experiment scripts: acquisition and baselines

notebooks/inspect_dataset.py

Purpose and data flow. Loads the dataset training split through Hugging Face and prints schema, sample daily rows and hourly arrays.

How the code works. Uses a project-local cache directory and `load_dataset(..., split="train")`. The loader may also download/cache evaluation files.

Why this design. Inspecting a real schema first establishes IDs, field types and array structure. Downloading evaluation files is distinct from using them to select a model.

notebooks/audit_dataset.py

Purpose and data flow. Audits cached training Arrow shards and writes reports/training_audit.json.

How the code works. `inspect_batch()` counts nulls, malformed hourly arrays, invalid values and daily/hourly inconsistencies. `summarize_series()` checks coverage, duplicates and gaps. `main()` verifies a complete shard set and records SHA-256 hashes.

Why this design. The audit reports evidence instead of silently repairing ambiguous data. Batch reading keeps large hourly data manageable; checksums make later changes detectable.

notebooks/baseline_forecast.py

Purpose and data flow. Builds the deterministic subset, evaluates the original last-14-day baseline window, and saves `baseline_metrics.json` plus processed Parquet files.

How the code works. `choose_pairs()` hashes first-day IDs; `verified_shards()` resolves audited training files inside the cache and checks hashes; `batches()` reads required scalar columns; `main()` checks 90-day coverage and reports segments.

Why this design. These helpers are reused by later runners so subset and data identity remain stable. The original single-window report is historical evidence, not the final selection report.

Why scripts are invoked as modules

```
python -m notebooks.baseline_forecast
```

Run from the repository root so imports of `demand_forecasting` resolve consistently. The notebooks directory currently contains executable Python scripts, not a requirement to run interactive `.ipynb` notebooks. `main` guards allow tests to import helper functions without launching the experiment.

Experiment scripts: cross-validation and diagnosis

[notebooks/cross_validate_baselines.py](#)

Purpose and data flow. Rebuilds the verified 200-series subset and writes `baseline_cross_validation.json` and `baseline_cross_validation_predictions.parquet`.

How the code works. Checks the selected-pair manifest and revision, then calls `evaluate_baselines()` to obtain fold and pooled results.

Why this design. Keeps the command-line orchestration separate from reusable evaluation logic. A single script recreates the reference benchmark on all three windows.

[notebooks/cross_validate_lightgbm.py](#)

Purpose and data flow. Fits one model per fold and writes LightGBM scores and row-level validation predictions.

How the code works. `load_verified_subset()` verifies hashes, IDs and coverage. `evaluate_lightgbm()` aligns baseline and feature frames, filters training by `history_ready`, and passes target-free inputs to prediction. `compare_metrics()` and `verify_baseline_parity()` check existing reference results.

Why this design. A fresh fit avoids carrying learned state between folds. Reference parity detects accidental changes in observations or evaluation arithmetic while adding a new candidate.

[notebooks/analyze_forecast_errors.py](#)

Purpose and data flow. Reads saved development predictions to diagnose errors by fold, series and stockout segment.

How the code works. `compare_methods()` calculates LightGBM absolute error minus baseline absolute error. `attach_stockout_labels()` uses a one-to-one join and verifies sales match the source. Outputs `forecast_error_analysis.json` and `forecast_errors_by_series.csv`.

Why this design. A positive `extra_absolute_error` means LightGBM is worse. This exposes concentrated losses hidden by an aggregate score. Target-date stockout labels are attached only retrospectively for analysis, not used as forecast inputs.

The diagnostics showed that improvements across many individual series did not necessarily outweigh losses on a few series. That motivated testing a simple fixed blend rather than assuming the more complex learner should win.

Experiment scripts: blends and alternative models

notebooks/evaluate_fixed_blend.py

Purpose and data flow. Adds the equal-weight blend to saved LightGBM and baseline validation predictions.

How the code works. Checks unique series/date keys, fold membership, exact pair/date coverage and reproduction of original metrics. Writes fixed_blend_cross_validation.json and fixed_blend_predictions.parquet.

Why this design. No retraining or blend-weight search is needed. Using the same saved rows makes the comparison controlled and prevents a changed population from masquerading as improvement.

notebooks/compare_models.py

Purpose and data flow. Compares linear regression, Ridge and logistic-plus-Ridge alongside the baseline, LightGBM and blend.

How the code works. fit_predict_linear_family() fits StandardScaler and OneHotEncoder only on each fold's training data. IDs and weekday are categorical for these models; other numeric features are scaled. Ridge uses alpha=1 and lsqr. Original candidates must reproduce recorded fold metrics.

Why this design. Linear models need explicit encoding and benefit from scaling. Different preprocessing is appropriate for each model family, but all candidates use the same forecast information and evaluation rows.

Why logistic regression is paired with Ridge

`expected sales = P(sales > 0) * estimated positive sales`

Logistic regression is a classifier, not a direct continuous-sales regressor. It estimates whether sales are positive. A Ridge model trained only on positive targets estimates the conditional amount. If training has only zeros or only positives, the probability path uses a constant instead of attempting an invalid single-class fit.

The script treats classifier convergence warnings as errors and clips negative quantity predictions. main() writes model_comparison.json and model_comparison_predictions.parquet. Parameters are fixed: this is a candidate comparison, not an exhaustive search or proof that the winning method is globally optimal.

The selected blend remained best in pooled development WAPE and each fold. Linear regression was weakest overall; switching algorithms alone did not deliver the desired 10-15% error.

Experiment scripts: final assessment and release

notebooks/check_final_evaluation.py

Purpose and data flow. Writes final_evaluation_data_check.json without fitting or scoring.

How the code works. Discovers evaluation Arrow files beside audited training shards, records hashes and schema, and checks dates, selected-series values and exact series/date coverage.

Why this design. Separate structural checking from scoring. The recorded cache hash is local provenance, not independently authenticated publisher proof.

notebooks/evaluate_final.py

Purpose and data flow. Fits once on 12,000 eligible training rows and scores 1,400 reserved observations.

How the code works. evaluate() rejects overlap and incomplete coverage, fits on training-only features, then uses shifted historical evaluation features. main() requires committed code, verifies data hashes and records pooled/daily scores plus prediction-file hash.

Why this design. The chosen method remains fixed regardless of results. Existing outputs are not silently overwritten. The output is final_evaluation.json and ignored final_evaluation_predictions.parquet.

notebooks/package_model.py

Purpose and data flow. Rebuilds the same training-only fit and exports a commit-named release directory.

How the code works. Checks final-report parameters, features, data revision, selected pairs and training coverage. Records source and report hashes, then writes model_package_build.json.

Why this design. Final scoring did not save its fitted object. Rebuilding needs verification against the recorded predictions; exporting alone is not enough.

notebooks/verify_model_package.py

Purpose and data flow. Verifies the real loaded package against every recorded final prediction.

How the code works. Checks package/report/prediction integrity and coverage. For each date, builds serving features from prior history, predicts, then appends that day's actual sales. Compares all three prediction columns with tight tolerances.

Why this design. Tests the actual serving route rather than only matching aggregate WAPE. model_package_verification.json records zero differences on all 1,400 rows; no model is retrained here.

Application scaffolding and legacy code

app/main.py and app/__init__.py

Purpose and data flow. Both files are currently empty scaffolding.

How the code works. There is no implemented FastAPI app or routed endpoint in this inspected checkout.

Why this design. Installing FastAPI and describing endpoints is not the same as implementing a service. Health, readiness, authentication and prediction routes must still be built.

app/schemas.py

Purpose and data flow. Contains the prototype PredictionRequest and PredictionResponse Pydantic models.

How the code works. Annotated and Field enforce finite nonnegative lag values; ConfigDict(strict=True, extra="forbid") rejects unsupported coercions and unexpected fields. Calendar fields have ranges; model_version must be usable text.

Why this design. This demonstrates input validation, but the contract is old: it asks callers for lags and month and labels outputs as demand. The selected model uses store/product IDs and rolling features. Do not wire this prototype directly to the new model.

```
Demand = Annotated[
    float, Field(strict=True, ge=0, allow_inf_nan=False)
]
# Validation constrains values, not their historical provenance.
```

The future API should accept identifiers and a forecast date, retrieve earlier history, and invoke the tested serving adapter. A numerical schema alone cannot establish that lag_7 actually came from seven days earlier.

notebooks/train_model.py

Purpose and data flow. Legacy synthetic demonstration that writes artifacts/model.joblib.

How the code works. generate_and_train() creates 365 artificial days with a sine pattern and noise, builds lag/calendar features, makes an 80/20 chronological split and fits a small LightGBM model.

Why this design. Useful as an initial learning scaffold, but it is not the FreshRetailNet training or release workflow. Its synthetic target, feature set and joblib artifact must not be confused with the selected blend.

Current active release: artifacts/releases/blend-v1-917dab07c759/. model.txt stores the native learner; metadata.json stores the serving contract and provenance. The baseline part of the blend is calculated from history, not stored as additional learned weights.

Tests: what each file protects

The latest reported complete run has 141 passing cases. Parameterized tests count multiple cases, so this is not the number of test files. This document records that run; it did not execute a fresh suite. Tests mainly use small synthetic fixtures, making checks fast and independent of downloading the full dataset.

Test file under tests/	Main behavior verified
test_schemas.py	Prototype request/response values, extra fields, calendar bounds and model-version validity.
test_audit_dataset.py	Hourly alignment, malformed arrays, daily/hourly mismatch and gaps hidden by duplicates.
test_baseline_forecast.py	Temporal boundaries, isolated series, stable selection, zero-total WAPE and changed-file rejection.
test_validation.py	Fold dates, shared calendars, shuffled inputs, duplicates, missing dates and insufficient history.
test_evaluation.py	Baseline fold counts, known errors, input ordering and future-change invariance.
test_features.py	Exact lag/window values, history readiness, future isolation and excluded targets.
test_modeling.py	Fresh fits, valid outputs, unknown pairs, training-cutoff rejection and target-free prediction.
test_lightgbm_evaluation.py	Separate fold fits, target exclusion and rejection of changed reference metrics/coverage.
test_error_analysis.py	Error-sign meaning and safe one-to-one diagnostic joins without changed source sales.
test_ensemble.py	Equal averaging; rejection of incompatible shapes, nonfinite and negative forecasts.
test_model_comparison.py	Finite outputs, single-class handling, ignored prediction targets and unknown pairs.
test_final_evaluation.py	One training-only fit, future-target invariance, lawful history updates, overlap/coverage errors.
test_serving.py	Offline/serving feature parity, shuffled history, non-mutation and invalid/stale history rejection.
test_artifacts.py	Export metadata, no overwrite, integrity rejection, contract validation and loaded prediction parity.

Tests establish the behaviors they exercise. They do not currently establish live API security, database access control, deployment latency, or backup recovery because those components are not implemented.

Environment, configuration and CI files

File	Function and reason
requirements/runtime.in	Direct libraries intended for inference: FastAPI, Uvicorn, Pydantic, pandas, NumPy, scikit-learn, LightGBM and joblib. Presence does not imply all are used by the active serving path.
requirements/training.in	Includes runtime dependencies plus datasets and pyarrow for acquisition and processing.
requirements/development.in	Includes training dependencies plus pytest, httpx, pip-audit, pip-tools and Ruff.
requirements/development-windows-py311.txt	Generated exact dependency versions and hashes for Windows/Python 3.11. Install with --require-hashes; regenerate from input files rather than editing manually.
requirements/README.md	Explains inputs, lock generation, clean installation and the unresolved Linux deployment lock.
requirements.txt; requirements-dev.txt	Legacy bootstrap lists. The documented verified Windows workflow uses the generated lock instead.
pyproject.toml	Ruff target/line length/exclusions, lint rules and pytest test discovery. Ruff formatting standardizes layout; it does not prove model correctness.
.gitignore	Excludes environments, caches, data, secrets/private-key patterns and generated releases. Ignoring a path does not untrack files already committed.
.github/workflows/checks.yml	Defines Windows locked installation, pip check, lint, formatting, tests and pip-audit; Linux installs from .in files and runs compatibility tests/audit.

Reproducibility boundaries

The explicit `.venv-check` interpreter avoids accidentally using another activated environment. Windows uses a hashed lock; Linux CI currently checks compatibility with resolved input dependencies and is not a fully locked deployment environment. Installer-tool upgrades in CI also mean the bootstrap stage is not completely pinned.

CI uses read-only contents permissions, disables persisted checkout credentials and sets job timeouts. pip-audit checks known dependency vulnerabilities; it does not scan application logic or guarantee safety. Secret scanning is a planned control, not a step visible in this workflow.

Reports, documents and generated files

File(s), relative to repository root	Meaning
reports/training_audit.json + .md	Machine-readable training checks/provenance and their human explanation.
reports/baseline_metrics.json; baseline_results.md	Original deterministic-subset, single-window baseline results.
reports/baseline_cross_validation.json + .md	Three-fold benchmark results and explanation.
reports/lightgbm_cross_validation.json + .md	LightGBM versus baselines on aligned folds, parameters and versions.
reports/forecast_error_analysis.json + .md; forecast_errors_by_series.csv	Retrospective stockout/series diagnostics; the CSV is derived diagnostics, not raw sales history.
reports/fixed_blend_cross_validation.json + .md	Fixed 50/50 development comparison.
reports/model_comparison.json	Six-method comparison with preprocessing, fold details and settings.
reports/final_evaluation_data_check.json	Evaluation schema, coverage and source hashes before scoring.
reports/final_evaluation.json + .md	Final metrics, protocol, implementation identity and interpretation.
reports/model_package_build.json	Export identity, model/metadata hashes; historical status says verification was pending.
reports/model_package_verification.json	Subsequent proof of 1,400-row parity; read alongside the build record.
reports/dependency_audit.json	Historical audit record; not proof that all current dependencies are vulnerability-free.
docs/project_scope.md; docs/model_selection.md	Business boundaries and frozen selection decision. Some scope status/checklist wording predates completed work.
README.md; DEVELOPMENT.md; SECURITY.md	Project introduction, original roadmap, and security guidance. Older API-first, synthetic or reserved-evaluation wording must be interpreted against current code/reports.

Local generated files: data/huggingface_cache contains cached source data; data/processed contains subsets and prediction Parquet files. artifacts/releases contains native packages. venv/, .venv-check/, __pycache__/, .pytest_cache/ and .ruff_cache/ are local runtime or cache directories, not project deliverables.

Security, saving work and resuming safely

Controls implemented so far

Data hashes and cache-path checks; strict temporal/coverage validation; finite/nonnegative value checks; trusted-hash artifact loading; exact dependency compatibility checks; ignored secret patterns; automated tests and dependency audits. These reduce specific risks. They do not make an unfinished service production-secure.

Controls still required

Database roles and network restrictions; parameterized queries; API authentication and HTTPS; request/rate/time limits; secret scanning and secret management; controlled release distribution; container hardening; operational logs, monitoring, backup/restore and rollback. Repository branch protection and live remote CI state were not independently verified for this PDF.

Git state versus unsaved editor tabs

Git sees files saved on disk, not unsaved VS Code buffers. A clean status can coexist with unsaved edits. Compare an uncertain buffer against its saved version before choosing Save or Don't Save. The model type-annotation fix is now merged and requires no model retraining.

```
git status --short --branch
git diff
# Save and review intended editor changes before committing.
# After a PR is merged and the working tree is clean:
git switch main
git pull --ff-only origin main
```

GitHub stores committed source and reports. Ignored data and release directories exist only on the local machine unless separately backed up. A merge is not a backup of those directories. Do not delete final predictions: package verification depends on their recorded hash and contents.

Routine checks when code changes

```
.\.env-check\Scripts\python.exe -m ruff check .
.\.env-check\Scripts\python.exe -m ruff format --check .
.\.env-check\Scripts\python.exe -m pytest -q
git diff --check
```

These are reference commands, not a request to rerun completed experiments now. Final scoring and package builders have overwrite guards; do not bypass them casually. Repeated evaluation after model changes does not create a new untouched holdout.

Remaining work and evidence index

Next stage	Reason and acceptance evidence
1. PostgreSQL foundation	Verify Docker/Compose, configure persistent local storage and restricted access. Separate migration, ingestion and serving privileges.
2. Schema and ingestion	Use reviewed migrations and validated, repeatable loads. Enforce unique series/date records; retain data provenance.
3. Historical retrieval	Queries must select observations strictly before forecast_date. Database-derived features must match the reference pipeline.
4. FastAPI implementation	Replace the prototype feature-only contract with identifiers/date inputs; load a trusted package once; add health/readiness, authentication and error handling.
5. Deployment and operations	Create a tested Linux dependency lock, container image and deployment process; verify API parity, limits, latency, monitoring and recovery.
6. Inventory simulation	State lead-time, inventory and uncertainty assumptions. Report simulated effects separately from observed-sales forecasting results.

Evidence used for this document

Local snapshot: main at b0dd1e2 (PR #14 merged). Core implementation: demand_forecasting/*.py. Experiment orchestration: notebooks/*.py. Outcome sources: reports/model_comparison.json, reports/final_evaluation.json, reports/model_package_build.json and reports/model_package_verification.json. Scope and decisions: docs/project_scope.md and docs/model_selection.md. Validation definitions: tests/*.py and .github/workflows/checks.yml.

Latest test count is taken from the user's successful terminal run (141 passed). Source files and reports were inspected read-only for this document. Neither training nor tests nor remote CI were rerun during authorship. Some older project documents lag behind the implementation; this guide separates current evidence from original plans.

Repository and dataset references

GitHub: [garietan3066/demand-forecasting-service](#)

Dataset: [Dingdong-Inc/FreshRetailNet-50K](#)

Dataset paper: [arXiv 2505.16319](#)

Resume point: modeling and verified packaging are complete. PostgreSQL implementation is paused and has not started. Continue with infrastructure verification when ready; do not restart model experimentation or recreate the completed release merely to resume development.